

LINKED LIST QUIZ

GATE Computer Science (CS/IT) Previous Year Concepts

Part I: Quiz Questions

Question 1

What is the worst-case time complexity to find the middle element of a singly linked list using the two-pointer (slow and fast) approach?

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n \log n)$

Question 2

Consider the following C function that takes a pointer to the head of a linked list:

```
void fun(struct Node* head) {  
    if(head == NULL) return;  
    fun(head->next);  
    printf("%d ", head->data);  
}
```

What does this function do?

- A. Prints all nodes of linked list in normal order.
- B. Prints all nodes of linked list in reverse order.
- C. Prints alternate nodes of the linked list.
- D. Results in an infinite loop.

Question 3

Which of the following operations is performed more efficiently by a Doubly Linked List than by a Singly Linked List?

- A. Deleting a node whose pointer is given.
- B. Searching for an unsorted element.
- C. Inserting a new node after a node whose pointer is given.
- D. Traversing the list from the beginning to the end.

Question 4

In a circular linked list, insertion of a node at the head of the list requires modification of how many pointers? (Assume we only have a head pointer, no tail pointer).

- A. 1
- B. 2
- C. $O(N)$ traversal is needed to find the last node, then modify 2 pointers.
- D. $O(1)$ time is required.

Question 5

Floyd's Cycle-Finding Algorithm uses two pointers moving at different speeds. If a cycle exists, what is the maximum number of times the fast pointer will traverse the loop before they meet?

- A. Exactly length of the loop
- B. It depends on the length of the non-cyclic part
- C. Less than or equal to the length of the loop (1 full loop max)
- D. Infinite times

Question 6

You are given a pointer to a node (not the tail node) in a singly linked list. You do not have the head pointer. What is the time complexity to delete this node?

- A. $O(N)$
- B. $O(1)$
- C. $O(N \log N)$
- D. It is not possible to delete the node without the head pointer.

Part II: Answers & Explanations

Answer 1

Correct Option: C. $O(n)$

Although the fast pointer moves at twice the speed of the slow pointer, it still traverses the 'n' elements of the list to reach the end. Therefore, the time complexity is strictly linear, $O(n)$.

Answer 2

Correct Option: B. Prints all nodes of linked list in reverse order.

This is a classic recursive function. It recursively calls itself with `head->next` before it executes the `printf` statement. This means the print statements are pushed onto the call stack and executed in LIFO (Last In, First Out) order, resulting in the list being printed backwards.

Answer 3

Correct Option: A. Deleting a node whose pointer is given.

To delete a node in a singly linked list, you need a pointer to its previous node to update the ``next`` pointer. Without head, finding the previous node takes $O(N)$. In a doubly linked list, you can instantly access the previous node via the ``prev`` pointer, making deletion $O(1)$.

Answer 4

Correct Option: C. $O(N)$ traversal is needed to find the last node, then modify 2 pointers.

To insert at the head of a circular linked list when you only have the ``head`` pointer, the last node's ``next`` pointer must be updated to point to the newly inserted head. To find the last node, you must traverse the entire list, taking $O(N)$ time.

Answer 5

Correct Option: C. Less than or equal to the length of the loop (1 full loop max)

Once both the slow and fast pointers are inside the loop, the distance between them decreases by 1 in each step. The maximum distance between them can be the length of the loop (L) minus 1. Thus, they will meet in at most $L-1$ steps, meaning the fast pointer does not complete more than one full cycle before catching the slow pointer.

Answer 6

Correct Option: B. $O(1)$

This is a classic trick question. You can copy the data from the *next* node into the current node, and then bypass (delete) the next node. Because you are copying data and updating one pointer (`node->next = node->next->next`), it takes $O(1)$ time. (Note: This fails if the node to be deleted is the very last node).